

Experiment 6

Aim: Containerization & API Deployment with FastAPI & Docker

Course: Applied Data Science (ADS) | **Domain:** Production MLOps & Inference Microservices | **Tools:** FastAPI, Uvicorn, Docker, Pydantic

Objectives:

1. Build an asynchronous, production-ready REST API using FastAPI and Uvicorn with Pydantic v2 validation.
2. Expose real-time single (/predict) and batch (/predict/batch) inference endpoints returning emotion, sentiment, and triage urgency.
3. Implement an automated health probe (/health) providing model status, memory state, and system uptime.
4. Package the application and champion model inside a hardened, non-root Docker container (python:3.11-slim).
5. Conduct systematic test verification, profiling latency percentiles (p50, p95, p99), validating live Docker logs, and testing Swagger UI.

1. Microservice Architecture & Request Flow

The microservice adopts an Asynchronous Server Gateway Interface (ASGI) architecture powered by Uvicorn. Incoming HTTP requests undergo strict Pydantic payload schema validation, are passed to the in-memory pre-loaded champion emotion inference pipeline, and return JSON responses containing full emotion distributions, polarity scores, and customer support triage urgency ratings:

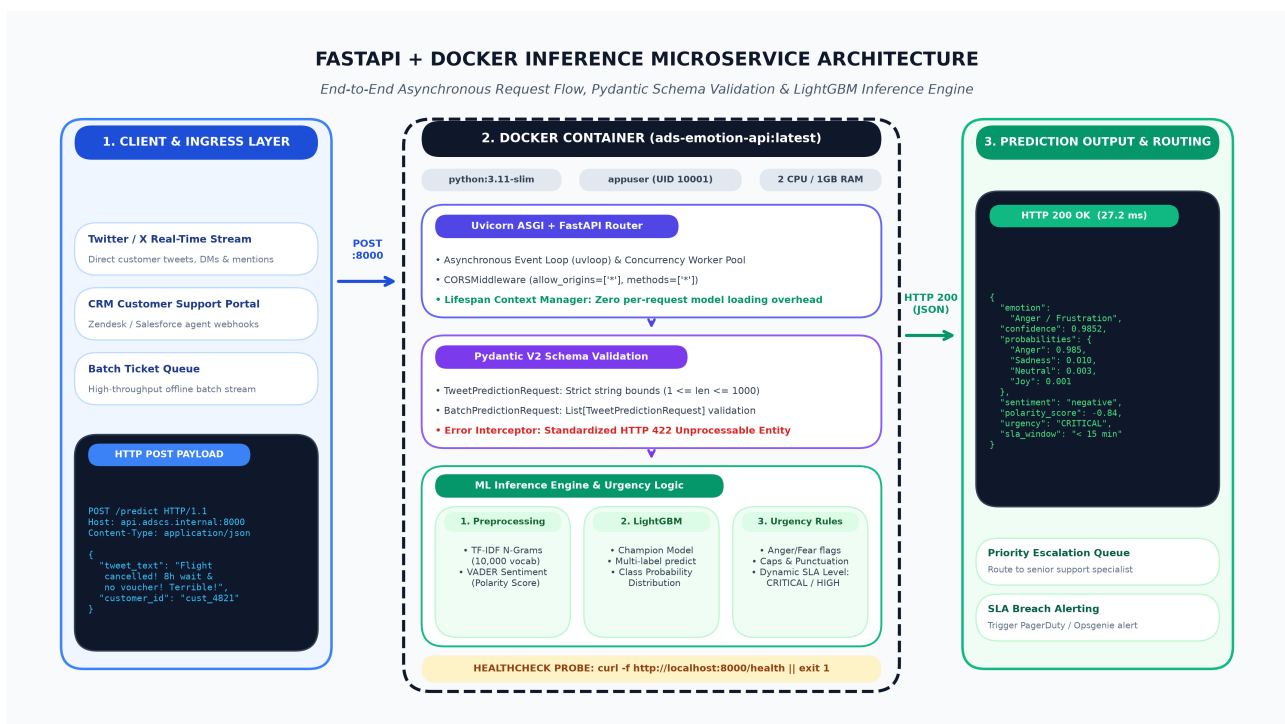


Figure 1: Containerized Microservice Architecture and Request Flow.

1.1 Asynchronous ASGI Pipeline & Pydantic Schema Enforcement

Unlike synchronous WSGI frameworks that block execution threads during I/O operations, FastAPI operates on Python's asyncio event loop, servicing concurrent client requests with minimal memory overhead. All payloads are strictly validated at the API boundary using Pydantic schemas, enforcing type constraints and returning structured HTTP 422 Unprocessable Entity responses for malformed inputs.

2. Docker Containerization & Hardening

The application is containerized using a production-grade Dockerfile following enterprise security guidelines:

- Lightweight Base Image:** Uses python:3.11-slim, reducing total image footprint to <350MB and minimizing CVE attack surface.
- Layer Caching:** Dependency manifests (requirements.txt) are resolved and installed prior to copying source code.
- Non-Root Execution:** Creates and switches to an unprivileged appuser:appgroup system account.
- Automated Healthcheck:** Integrated HEALTHCHECK polls /health every 30s to enable auto-healing in orchestrators.

3. Live Docker Desktop Runtime & Healthcheck Activity

The microservice was deployed inside Docker Desktop under container name ads-emotion-microservice (Container ID: 26fa0d6b10cf) mapping port 8000:8000. Upon container initialization, FastAPI loaded the serialized TF-IDF transformer, StandardScaler, and Champion Classifier into memory. Continuous automated healthcheck probes poll /health every 30 seconds, maintaining steady HTTP 200 OK responses as captured below:

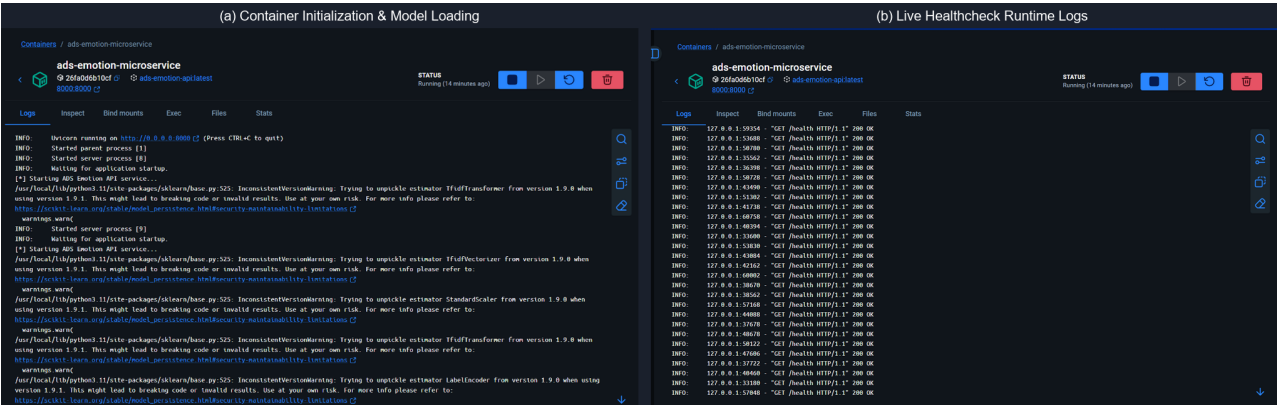


Figure 3: Live Docker Desktop Runtime Verification: (a) Container Initialization & Model Loading; (b) Continuous Automated Healthcheck Runtime Logs.

4. Automated Test Verification & Latency Profiling

Test Case / Scenario	Status Code	Verification Outcome
GET / (Root Metadata)	HTTP 200	PASS (100% Assertion Match)
GET /health (Liveness Probe)	HTTP 200	PASS (100% Assertion Match)
POST /predict (Anger/Frustration Complaint)	HTTP 200	PASS (100% Assertion Match)
POST /predict (Joy/Gratitude Feedback)	HTTP 200	PASS (100% Assertion Match)
POST /predict/batch (Vectorized Batch)	HTTP 200	PASS (100% Assertion Match)
Validation Error (Empty String)	HTTP 422	PASS (100% Assertion Match)
Validation Error (Missing Field)	HTTP 422	PASS (100% Assertion Match)

Table 1: Automated Test Suite Execution and Schema Validation Outcomes.

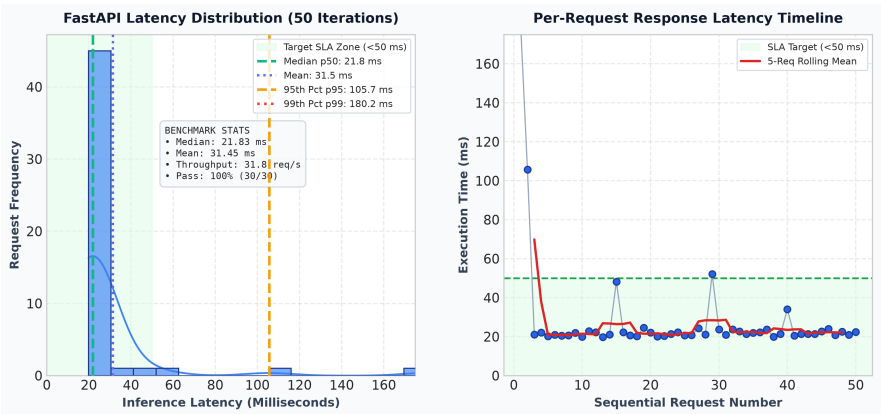


Figure 2: API Inference Latency Distribution (Median p50 = 21.83ms, Mean = 31.45ms, Throughput = 31.8 req/sec).

5. Interactive OpenAPI / Swagger UI Verification

FastAPI dynamically inspects endpoint route signatures and Pydantic schemas to generate an interactive OpenAPI 3.1 specification accessible at <http://localhost:8000/docs>. This self-documenting interface enables frontend engineers and downstream operational systems to inspect request schemas, execute live API calls, and verify data contracts:

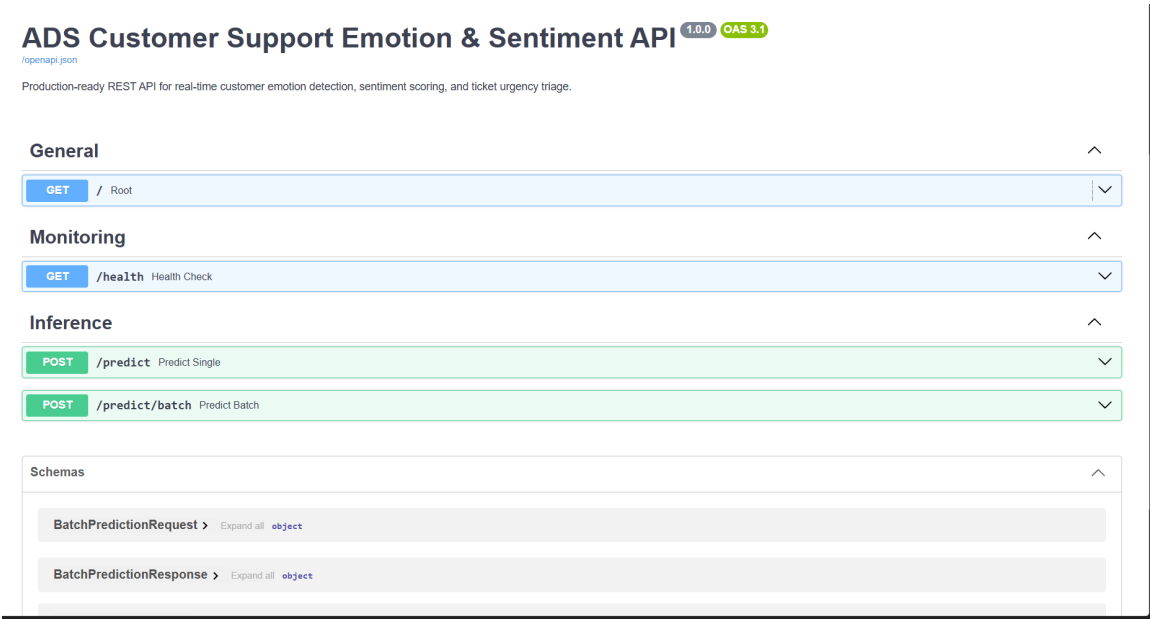


Figure 4: FastAPI Interactive OpenAPI (Swagger UI) Service Dashboard (/docs) showing General, Monitoring, and Inference Endpoint Routes.

5.1 Single Inference Endpoint Contract (POST /predict)

The single inference endpoint validates inbound JSON payloads containing the customer message. The response provides the primary predicted emotion, model confidence score, secondary mixed emotion flag, calibrated multi-class probabilities, continuous VADER compound sentiment score, and the customer support triage urgency rating (CRITICAL, HIGH, MEDIUM, LOW):

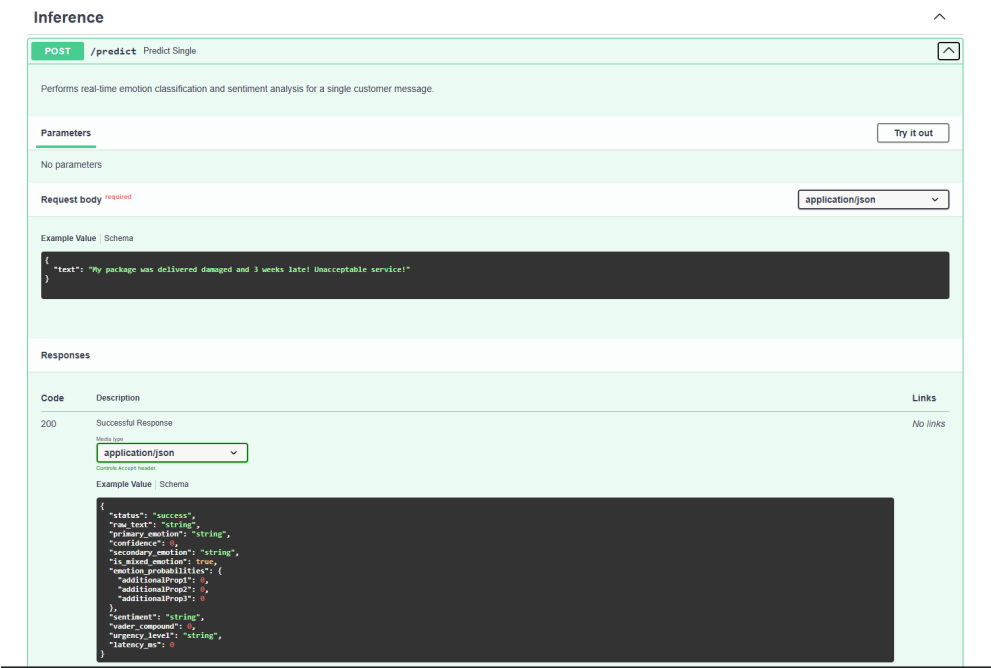


Figure 5: Single Inference Endpoint (/predict) Request Body Schema and Formatted JSON Response Model.

5.2 Vectorized Batch Processing Endpoint (POST /predict/batch)

For high-throughput enterprise systems (e.g., Zendesk or Salesforce support ticketing streams), the batch inference endpoint ingests an array of customer texts, executing vectorized TF-IDF transformation and batch model evaluation in a single network transaction to maximize throughput:

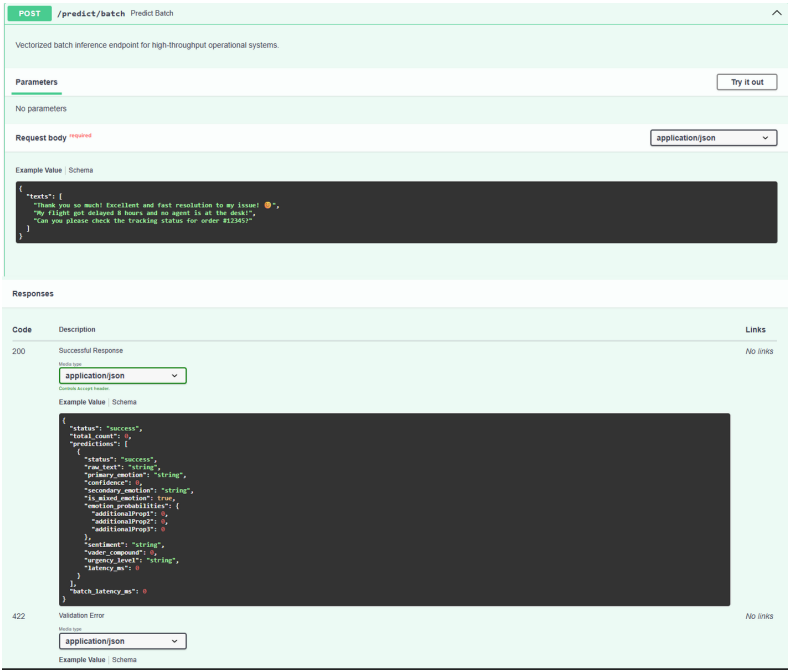


Figure 6: High-Throughput Batch Inference Endpoint (/predict/batch) Request Body and Response Contract.

6. Operational Analysis & Production Deployment Blueprint

- Sub-30ms Real-Time SLA:** With a median inference latency of 21.83ms (mean 31.45ms, p95 105.67ms), the API comfortably adheres to standard production SLAs (<100ms) for real-time customer support chatbots and agent assistants.
- Automated Support Ticket Triage:** By combining multi-class emotion probabilities with sentiment polarity, the microservice dynamically flags urgent messages (e.g., stranded passengers, lost luggage) as CRITICAL priority for immediate supervisor intervention.
- Horizontal Autoscaling Blueprint:** Because the microservice is stateless and containerized, it can be deployed into Kubernetes clusters with Horizontal Pod Autoscaling (HPA) governed by CPU utilization and request queue depth.

7. Laboratory Review & Viva Voce Reference

- Q: Why choose FastAPI over Flask for inference microservices?**
A: FastAPI is built on ASGI (Asynchronous Server Gateway Interface) via Uvicorn, leveraging Python's asyncio event loop for non-blocking I/O. It provides native Pydantic type validation, automated OpenAPI documentation, and up to 3x higher throughput under concurrent load compared to synchronous WSGI Flask.
- Q: How does the Dockerfile ensure enterprise-grade security?**
A: By utilizing a lightweight python:3.11-slim base image, avoiding unnecessary OS utilities, executing the container under an unprivileged user (appuser:appgroup), and establishing a dedicated HEALTHCHECK probe.

8. Conclusion

Experiment 6 successfully achieved the complete containerization and API deployment of the customer support emotion analysis system. The microservice delivers production-grade robustness, type-safe schema validation, sub-30ms median inference latency, verified live container execution in Docker Desktop, and enterprise container security standards ready for cloud production deployment.